

WhatsApp Expense Management System

System Documentation

1. Overview

The **WhatsApp Expense Tracker** is an expense-management system that allows users to record shared expenses and payments through WhatsApp and view the resulting financial data through a web-based dashboard.

The system consists of four main components:

1. **WhatsApp** — receives expense and deletion commands.
 2. **n8n Workflow** — processes WhatsApp messages, uses an AI model to extract structured information, generates Transaction IDs, splits expenses, and writes records to Google Sheets.
 3. **Google Sheets** — stores Expenses and Payments records.
 4. **Google Apps Script + HTML Dashboard** — reads data from Google Sheets and provides a web interface for viewing and deleting transactions.
-

2. System Architecture

The overall workflow is:

WhatsApp Message



WhatsApp Trigger in n8n



Delete Command Detection



AI Message Parser



Split Into Rows



Expense / Payment Routing



Google Sheets



Apps Script



Web Dashboard

For deletion:

WhatsApp / Dashboard Delete Request

↓

Transaction ID

↓

Find Expense Rows + Find Payment Row

↓

Build Deletion Plan

↓

Delete Matching Rows

↓

Confirmation

The n8n workflow uses separate routing for Expense and Payment records.

3. Google Sheets Database

The system uses a Google Spreadsheet containing two primary sheets:

3.1 Expenses Sheet

The Expenses sheet stores individual expense-share records.

The expected fields are:

Field	Description
ID	Transaction ID
Date	Expense date
Category	Expense category
Person	Person responsible for the expense share
Amount	Individual share amount
Detail	Description of the expense
Sender Number	WhatsApp sender number

The n8n workflow maps these values when appending an expense record to Google Sheets.

Multiple Expense Rows

One transaction can generate **multiple expense rows**.

For example:

A total expense of Rs. 3,000 may be split between:

- Faheem — Rs. 1,000
- Amir — Rs. 1,000
- Hussain — Rs. 1,000

All three rows receive the **same Transaction ID**.

This is important because the Transaction ID represents the complete transaction rather than an individual expense row.

4. Payments Sheet

The Payments sheet stores the person who paid the total amount upfront.

The main fields are:

Field	Description
ID	Transaction ID
Date	Transaction date
Category	Expense category or settlement
Paid By	Person who paid
Total Amount	Total amount paid
Description	Description
Sender Number	WhatsApp sender number

The n8n Payment node maps these fields before appending the record to Google Sheets.

5. Transaction ID System

Each normal WhatsApp transaction receives a unique Transaction ID.

The n8n Code node generates the ID using the current date and a random four-character component.

The format is:

TXN-YYMMDD-XXXX

Example:

TXN-260911-A7K2

The same Transaction ID is assigned to:

- all Expense rows generated from the message
- the corresponding Payment row

Therefore, a single transaction can be identified across both sheets using one ID.

6. WhatsApp Message Processing

The workflow starts with the **WhatsApp Trigger** node, which receives incoming WhatsApp messages.

The message body is passed to the AI model for structured parsing.

The AI parser extracts information in the following structure:

```
{
  "category": "string",
  "detail": "string",
  "date": "YYYY-MM-DD or null",
  "total": 0,
  "paid_by": "string",
  "splits": [
    {
      "person": "string",
      "amount": 0
    }
  ],
  "isSettlement": false
}
```

7. Expense Categories

The AI parser recognizes three main expense categories:

Food

Examples include:

- Lunch
- Dinner
- Breakfast
- Tea

- Chai
- Coffee
- Cold drinks
- Juice
- Water
- Snacks
- Pizza
- Burger
- Biryani
- Groceries
- Toffee
- Candy
- Chocolate
- Biscuits
- Cigarettes
- Chips
- Namkeen
- Paan
- Gutka

Travel

Examples include:

- Petrol
- Fuel
- Taxi
- Bus
- Train
- Ticket
- Hotel stay
- City-to-city travel

Other

Used when the expense clearly does not belong to Food or Travel, such as:

- Furniture
- Bills
- Medicine
- Repairs

The category is stored as a lowercase single-word value.

8. Person Name Normalization

The system maintains a canonical list of supported names.

Examples include:

- Faheem
- Amir
- Hussain
- Shehzad
- Umair
- Abdul Rahim
- Sakhawat
- Shahid
- Farhat
- Khuram
- Salman
- Waheed
- Asif
- Majid
- Ghulam Mustafa
- Ahmad
- ShehzadMultan
- Bilal
- Fateh
- Babar
- Wajahat
- Mohsin

Names are normalized so that different capitalization or extra spaces do not create separate people.

For example:

faheem
FAHEEM
Faheem
fAhEeM

all become:

Faheem

The n8n workflow applies the same normalization to expense participants and the person who paid.

9. Expense Splitting

The **Split Into Rows** Code node converts the AI output into individual database records.

For every person in the `splits` array, an Expense row is created.

Each row contains:

- Transaction ID
- Record Type
- Date
- Category
- Person
- Amount
- Detail
- Sender Number

The record type is set to:

expense

The generated Transaction ID remains the same for all split rows belonging to the transaction.

10. Payment Recording

For every transaction, the workflow also creates a Payment record.

The Payment record contains:

- Transaction ID
- Record Type
- Date
- Category
- Paid By
- Total Amount

- Detail
- Sender Number

The record type is:

payment

The Expense and Payment records are then routed through the Switch1 node.

11. Settlement / Balance Cleared

The system supports settlement messages.

A settlement means that an existing balance is being cleared rather than creating a new expense.

Examples of settlement-related phrases include:

- balance cleared
- balance clear
- cleared balance
- clear balance
- balance settled
- settlement
- settled
- balance paid
- paid balance

When the AI identifies a settlement:

- `isSettlement` becomes true
- `splits` becomes an empty array
- `category` becomes settlement
- `detail` becomes Balance Cleared
- the person clearing the balance becomes `paid_by`
- the specified amount becomes the payment amount

No Expense row is created.

Only a Payment record is created.

The confirmation message also explicitly informs the user that the payment was recorded without booking a new expense.

12. WhatsApp Confirmation

After a transaction is successfully recorded, the workflow sends a confirmation message to the WhatsApp sender.

For a normal expense, the confirmation contains:

- Transaction ID
- Date
- Category
- Detail
- Total
- Paid By
- Split details

For a settlement, the confirmation contains:

- Transaction ID
- Date
- Amount
- Paid By
- Settlement status

The confirmation is generated by the **Send Confirmation** node.

13. Delete Transaction Feature

The system supports deleting a complete transaction using its Transaction ID.

A transaction can contain:

- one Payment row
- one or multiple Expense rows

Therefore, deletion must remove **all rows belonging to the same Transaction ID**.

The n8n workflow implements this using:

1. Is Delete Command
2. Extract Delete Info
3. Find Expense Rows

4. Find Payment Row
5. Build Deletion Plan
6. Anything To Delete?
7. Route By Sheet
8. Delete Expense Rows
9. Delete Payment Row
10. Merge Deletions
11. Finalize Deletion
12. Send Delete Confirmation

The workflow checks both the Expenses and Payments sheets for the supplied ID.

14. Delete Command

The workflow identifies a WhatsApp message as a deletion command when the message starts with:

`delete`

The Transaction ID is then extracted from the message.

The expected format is:

`delete TXN-XXXXXX-XXXX`

The extraction code searches for a Transaction ID beginning with TXN-.

15. Finding Records for Deletion

The workflow searches the Payments sheet using the Transaction ID.

It also searches the Expenses sheet using the same Transaction ID.

The Payment lookup uses the ID column, while the Expense lookup also uses the ID column.

This allows the system to find:

- the Payment row
 - all matching Expense rows
-

16. Safe Multiple-Row Deletion

A transaction may have multiple Expense rows.

The deletion workflow therefore sorts Expense rows by their spreadsheet row number in descending order before deletion.

For example:

Row 15
Row 20
Row 25

will be deleted in this order:

Row 25
Row 20
Row 15

This prevents row-number shifting from causing the wrong rows to be deleted.

The Build Deletion Plan node specifically implements this approach.

17. Delete Routing

The Route By Sheet switch separates deletion requests into two paths:

Expense

→ Delete Expense Rows

Payment

→ Delete Payment Row

The workflow then merges the deletion results before sending the final confirmation.

18. Delete Confirmation

After successful deletion, WhatsApp receives a confirmation containing:

- Transaction ID
- Payment information, when available
- Expense split information, when available

- Confirmation that the transaction was deleted from the sheets

The confirmation is generated by the **Send Delete Confirmation** node.

If no matching records are found, the system sends a separate “Entry Nahi Mili” message.

19. Google Apps Script

The Google Apps Script provides the backend for the web dashboard.

The Spreadsheet ID is configured as:

```
const SPREADSHEET_ID = '1p4CSRNPYrimZZ3F2S5fMLS5uuXLzm3dWtI_N3LeAp-w';
```

The script exposes the HTML dashboard through `doGet()`.

It loads the `index` HTML file and sets the application title to:

Expense Tracker

20. Reading Expense Data

The Apps Script function:

```
getPersonDataPublic(person)
```

reads both:

- Expenses
- Payments

from the configured spreadsheet.

Instead of depending on fixed column numbers, the script searches for column headers.

For example, it can identify:

- ID
- Transaction ID
- Date
- Category
- Person
- Amount
- Detail

This makes the data-reading logic less dependent on the exact column order.

21. Person-Based Dashboard

The dashboard supports selecting:

- an individual person
- All Persons

When an individual is selected, the Apps Script filters the records based on the person's name.

When:

__ALL__

is selected, the dashboard combines records for all people.

The dashboard calculates:

- Total Expense
 - Total Paid
 - Net Balance
 - Record Count
 - Category Totals
-

22. Balance Calculation

The system calculates the balance using:

Balance = Total Paid - Total Expense

Therefore:

Positive Balance

The person has paid more than their recorded expenses.

Negative Balance

The person's recorded expenses are greater than the amount they paid.

The HTML dashboard displays the balance using different visual classes for positive and negative values.

23. Person List

The Apps Script function:

```
getPersonListPublic()
```

collects people from both:

- Expenses → Person
- Payments → Paid By

Duplicate names are removed using a JavaScript Set.

Names are then normalized into proper case and sorted alphabetically.

The dashboard loads these names into the person-selection dropdown.

24. Web Dashboard

The dashboard is a mobile-friendly HTML application.

Its main sections are:

1. Person Selection
2. Delete Transaction
3. Summary Cards
4. Recent Expenses
5. Category Breakdown

The interface is designed around a maximum width of approximately 520px, making it suitable for mobile screens as well as desktop browsers.

25. Summary Cards

The dashboard displays four primary metrics:

Total Expense

Total expense amount for the selected person or all persons.

Total Paid

Total amount paid by the selected person or all persons.

Net Balance

Difference between paid and expense amounts.

Records

Number of expense records displayed.

The dashboard updates these values whenever the user selects a person and clicks **View**.

26. Recent Expenses

The dashboard displays recent expense records.

Each record can show:

- Category
- Detail
- Transaction ID
- Date
- Person when viewing all persons
- Amount

The dashboard limits the displayed list to the first 100 records returned by the Apps Script.

27. Category Breakdown

The dashboard calculates category totals from the expense records.

Categories are sorted by their total amount in descending order.

Each category displays:

- Category name
- Combined category total
- Amount

This provides a quick overview of where money is being spent.

28. Dashboard Delete Feature

The dashboard also provides a direct Transaction ID deletion interface.

The user enters a Transaction ID into the Delete Transaction field and clicks:

Delete

Before deletion, the dashboard displays a confirmation dialog.

The confirmation warns that all Expense and Payment records with that Transaction ID will be deleted.

The HTML then calls:

```
google.script.run.deleteTransactionPublic(transactionId);
```

29. Apps Script Transaction Deletion

The Apps Script function:

```
deleteTransactionPublic(transactionId)
```

handles dashboard-based deletion.

It checks both:

- Expenses
- Payments

for the supplied Transaction ID.

The function identifies the Transaction ID column by header name and deletes matching rows.

Importantly, deletion is performed from the bottom of the sheet toward the top. This ensures that deleting multiple matching rows does not cause row-number shifting problems.

The function returns:

```
{  
  transactionId: id,  
  deletedExpenses: deletedExpenses,  
  deletedPayments: deletedPayments,  
  message: 'Transaction ... deleted successfully.'  
}
```

30. Dashboard Refresh

After successful deletion, the dashboard:

1. Displays the number of deleted Expense rows.
2. Displays the number of deleted Payment rows.
3. Clears the Transaction ID input.
4. Reloads the dashboard data.

This means the displayed totals and recent records are immediately refreshed after deletion.

31. Error Handling

The system contains error handling at several levels.

Dashboard

If data loading fails, the dashboard displays an error message.

If deletion fails, the dashboard displays the returned error.

Apps Script

The script throws an error if:

- Expenses sheet is missing
- Payments sheet is missing
- Transaction ID is empty
- Transaction ID column cannot be found
- No matching transaction exists

n8n

The workflow sends a “not found” message when a requested Transaction ID does not exist.

32. Data Flow Example

Suppose the user sends:

```
Lunch 3000  
Faheem 1000  
Amir 1000
```

Hussain 1000
Paid by Faheem

The AI converts the message into structured information.

The system generates:

TXN-260911-A7K2

The Expenses sheet receives:

ID	Person	Amount
TXN-260911-A7K2	Faheem	1000
TXN-260911-A7K2	Amir	1000
TXN-260911-A7K2	Hussain	1000

The Payments sheet receives:

ID	Paid By	Total Amount
TXN-260911-A7K2	Faheem	3000

The Transaction ID therefore links the complete transaction across both sheets.

33. Complete Transaction Lifecycle

A normal transaction follows this lifecycle:

Step 1 — User Sends WhatsApp Message

The WhatsApp Trigger receives the message.

Step 2 — AI Parses Message

The AI extracts:

- category
- detail
- date
- total
- paid_by
- splits
- settlement status

Step 3 — Transaction ID Generated

The workflow generates one Transaction ID.

Step 4 — Expense Rows Created

One row is created for each person in the split.

Step 5 — Payment Row Created

One Payment row is created for the person who paid.

Step 6 — Google Sheets Updated

Both records are appended to their respective sheets.

Step 7 — WhatsApp Confirmation

The user receives the Transaction ID and transaction details.

Step 8 — Dashboard

The Apps Script dashboard reads the latest Google Sheet data.

Step 9 — Optional Deletion

The user can delete the complete transaction using the Transaction ID.

34. Key Design Principle

The most important design principle of this system is:

One Transaction ID represents one complete financial transaction.

That means the Transaction ID is shared by:

- one Payment record
- zero or more Expense records

This design makes it possible to delete an entire transaction even when the transaction contains multiple expense-split rows.

35. Components Summary

Component	Main Responsibility
-----------	---------------------

Component	Main Responsibility
WhatsApp	User interaction
WhatsApp Trigger	Receive messages
AI Model	Parse natural-language expense messages
Split Into Rows	Convert parsed data into database rows
Switch1	Route Expense/Payment records
Google Sheets	Store financial records
Apps Script	Dashboard backend and data processing
HTML Dashboard	View and delete transactions
Transaction ID	Link all records belonging to one transaction

36. Current Features

The current implementation supports:

- WhatsApp expense entry
- AI-based expense parsing
- Expense splitting between multiple people
- Payment recording
- Settlement / balance-cleared entries
- Canonical person-name normalization
- Automatic Transaction ID generation
- Google Sheets storage
- Person-based filtering
- All-person combined view
- Total Expense calculation
- Total Paid calculation
- Net Balance calculation
- Category breakdown
- Recent expense history
- Transaction ID display
- Transaction deletion from dashboard
- Transaction deletion through WhatsApp
- Multiple Expense rows deletion for one Transaction ID
- Payment row deletion
- Delete confirmation
- Not-found handling

- Automatic dashboard refresh
-

37. Important Operational Notes

The system depends on consistent sheet names and headers.

The Apps Script expects the following sheets:

Expenses

Payments

The Transaction ID column is identified through headers such as:

ID

Transaction ID

TransactionID

The system also relies on the Transaction ID being stored consistently in both Expenses and Payments.

If the ID is missing or inconsistent between the two sheets, complete transaction-level deletion may not work as intended.

38. Security and Access Considerations

The system contains sensitive financial records and WhatsApp sender information.

The Google Sheet, Apps Script deployment, n8n workflow, WhatsApp credentials, and OpenAI credentials should therefore be kept private and should not be shared publicly.

The dashboard should also be deployed with appropriate access restrictions if the expense data is not intended to be public.

39. Maintenance

For future modifications, the following areas should be considered together:

Adding a new Expense field

Update:

1. Google Sheets Expenses columns

2. n8n Expense mapping
3. Apps Script data-reading logic
4. HTML dashboard if the field needs to be displayed

Adding a new Payment field

Update:

1. Google Sheets Payments columns
2. n8n Payment mapping
3. Apps Script Payment-reading logic
4. HTML dashboard if required

Changing Transaction ID format

Update:

1. Transaction ID generation in n8n
2. Delete ID extraction logic
3. Any validation or display logic

Changing deletion behavior

Both deletion systems should be considered:

- n8n WhatsApp deletion
 - Apps Script dashboard deletion
-

40. Conclusion

The WhatsApp Expense Tracker is designed around a **Transaction ID–based financial record system**.

WhatsApp provides the input interface, n8n handles intelligent message processing and database writing, Google Sheets acts as the data store, and the Apps Script/HTML application provides a dashboard for reviewing and managing the stored data.

The Transaction ID serves as the central link between Expense and Payment records, allowing transactions with multiple expense splits to be treated as one complete transaction and deleted as a single unit.